



Optimization of Service Function Chain Migration Based on Graph Neural Networks and Deep Reinforcement Learning

Mengxue Liu¹, Hefei Hu^{2*}, Jing Ran¹

¹ School of Electronic Engineering, Beijing University of Posts and Telecommunications, 100876 Beijing, China

² School of Information and Communication Engineering, Beijing University of Posts and Telecommunications, 100876 Beijing, China

* Correspondence: Hefei Hu (huhefei@bupt.edu.cn)

Received: 11-20-2025

Revised: 02-10-2026

Accepted: 03-11-2026

Citation: M. X. Liu, H. F. Hu, and J. Ran, “Optimization of service function chain migration based on graph neural networks and deep reinforcement learning,” *Acadlore Trans. Mach. Learn.*, vol. 5, no. 2, pp. 153–166, 2026. <https://doi.org/10.56578/ataiml050205>.



© 2026 by the author(s). Licensee Acadlore Publishing Services Limited, Hong Kong. This article can be downloaded for free, and reused and quoted with a citation of the original published version, under the CC BY 4.0 license.

Abstract: In mobility-aware scenarios such as vehicular networks, mobile augmented reality (AR)/virtual reality (VR) services, and other latency-sensitive Multi-access Edge Computing (MEC) applications, continuous user movement leads to frequent migrations of service function chains (SFCs). Traditional approaches typically rely on global deployment comparisons, which fail to accurately identify the specific virtual network functions (VNFs) that require migration and their optimal target nodes. This limitation often results in redundant migrations, inefficient resource utilization, and an increased risk of service disruption, thus hindering the balance between latency assurance and resource efficiency. To overcome these limitations, this paper proposed a graph-enhanced deep reinforcement learning–based adaptive migration optimization (DRL-GAMO) framework. By integrating the topological representation capability of graph neural networks (GNNs) with the decision-making efficiency of deep reinforcement learning (DRL), DRL-GAMO established a topology–resource–decision mapping that jointly optimized VNF selection and determination of target nodes. This pre-migration decision process effectively reduced redundant operations and directed migration behaviors toward resource-efficient strategies. The designed reward function minimized migration overhead under service-level agreement (SLA) latency constraints and penalized downtime to maintain service continuity. Simulation results demonstrated that DRL-GAMO achieved stable service latency, lower resource consumption, and shorter migration time while reducing migration volume by more than 40% compared with DRL-ADMO, thereby improving the migration success rate and validating its effectiveness in MEC environments.

Keywords: Service function chain migration; Graph neural network; Deep reinforcement learning; Multi-access edge computing; Virtual network functions

1 Introduction

In mobility-aware scenarios, the initial deployment of service function chains (SFCs) should align with the network state and service requirements at the time of user access [1]. By sequentially placing virtual network functions (VNFs) on appropriate edge nodes, the initial latency constraints could be satisfied [2]. However, continuous user mobility dynamically alters the source node of an SFC as a change in access locations, hence significantly increasing the frequency and complexity of migration events. Existing migration methods remain constrained by limited node-selection flexibility and low scheduling efficiency, rendering them unsuitable for highly dynamic environments [3, 4].

Typical mobility-aware Multi-access Edge Computing (MEC) applications include vehicular networks (V2X), mobile AR/VR services, and real-time video analytics. These services are characterized by frequent handovers, dynamically shifting SFC entry points and strict end-to-end latency requirements, which enable efficient SFC migration to become essential for maintaining the continuity of service. However, MEC services exhibit diverse and stringent performance requirements: V2X demands ultra low latency and high reliability, AR/VR requires stable delay and high bandwidth, and video analytics emphasizes sustained throughput. Existing migration methods could not effectively accommodate these heterogeneous needs, as their fixed node-selection rules and coarse scheduling policies fail to balance latency, bandwidth, and resource constraints under dynamic mobility.

To address these challenges, several approaches have been examined [5]. Early optimization and heuristic methods, such as the Follow-Me Chain algorithm proposed by Chen and Liao [6], employ integer programming and online decision mechanisms to mitigate service interruptions caused by mobility [7]. Nevertheless, these methods rely on pre-defined rules, which restrict their adaptability to fluctuating network conditions. Subsequently, prediction-driven mechanisms were developed. For instance, Hu et al. [8] introduced the PathLoad Adaptive Routing and Bandwidth Allocation (PLARBA) algorithm, which employs soft migration and path load-adaptive scheduling to reduce migration failures, while Tang et al. [9] applied federated learning to predict VNF resource demands and optimize migration strategies, thereby mitigating service disruptions and resource waste [10]. However, the performance of such predictive approaches depends heavily on the availability of large-scale historical datasets, leading to degraded accuracy in data-sparse scenarios [11]. More recently, deep reinforcement learning (DRL) has emerged as a promising paradigm for dynamic SFC management [12, 13]. Xu et al. [14] proposed DRL-ADMO, which formulates migration as a Markov Decision Process (MDP) to enable intelligent parallel migration of multiple VNFs, while Vieira et al. [15] explored proactive migration by incorporating user trajectory and prediction of latency trend to maintain the quality of service (QoS) with minimal cost. However, these studies did not fully address coordination of resources among multiple VNFs, which may lead to contention and suboptimal utilization [16, 17].

Building upon these studies, this work introduced the graph-enhanced deep reinforcement learning-based adaptive migration optimization (DRL-GAMO) framework. The proposed method integrated a graph neural network (GNN) to jointly encode network topology and resource states, enabling topology-aware evaluation of node deployability that filtered feasible candidate nodes and effectively narrowed the decision space [18]. Furthermore, a Double Deep Q-Network (DDQN) was employed to jointly optimize migration strategies and resource mapping, thus achieving adaptive and efficient SFC migration under dynamic mobility. Experimental results on the SFCSim platform demonstrated that, under stringent latency constraints, DRL-GAMO markedly improved migration success rates and reduced both migration time and consumption of resources, while maintaining stable service latency. These results validated the robustness, scalability, and practical effectiveness of DRL-GAMO in MEC environments.

Based on the aforementioned motivation and design objectives, the following section presents the network model and formulation of mathematical problem that form the theoretical foundation of the proposed DRL-GAMO framework.

2 Network Model and Formulation of the Problem

2.1 Network Model and Virtual Network Functions

In a mobility-aware MEC environment, the network function virtualization infrastructure (NFVI) is modeled as an undirected graph $G^S = (N^S, E^S)$ [19], where N^S denotes the set of physical nodes, consisting of access nodes N_A^S and edge computing nodes N_E^S . Each edge node $n_i^s \in N_E^S$ provides limited computing resources $cap^{res}(n_i^s)$ of type $res \in R$ (e.g., CPU, memory, and storage). The physical links E^S connect adjacent nodes, where each link e_{ij}^s is associated with bandwidth $bw(e_{ij}^s)$ and propagation delay $delay(e_{ij}^s)$ [20].

To evaluate end-to-end transmission delay, the K -shortest path set between nodes n_p^s and n_q^s is denoted by $\tau_{pq} = \{\tau_{pql}\}$. Let $\sigma_{pq}^{ij} = 1$ if link e_{ij}^s is on path τ_{pql} , and 0 otherwise. The total delay of path τ_{pql} is expressed as

$$d_{pql} = \sum_{e_{ij}^s \in E^S} \sigma_{pq}^{ij} \cdot delay(e_{ij}^s) \quad (1)$$

All candidate routing paths form the set $\Gamma = \{\tau_{pq}\}$, which will be used for subsequent SFC deployment and migration.

Each VNF $f_i \in F$ represents a software-based network function executed on edge nodes. Due to heterogeneous resource requirements, the resource coefficient $coef^{res}(f_i)$ denotes the amount of resource type res consumed by f_i per unit of traffic. Additionally, the traffic scaling factor $ratio(f_i)$ captures how f_i modifies traffic volume during processing (e.g., encapsulation, compression, and filtering).

2.2 User Services and User Mobility

In mobility-aware MEC scenarios, dynamic changes in user access locations increase the distance between users and the deployed SFC nodes, resulting in higher transmission latency and potential network congestion. When the QoS requirements can no longer be satisfied, partial VNFs must be migrated to alternative nodes while meeting resource and bandwidth constraints.

The fundamental challenge of SFC migration lies in accurately identifying migration triggers and efficiently selecting suitable target nodes and routing paths. As users move across adjacent access nodes, handover events are generated, hence causing the SFC source node to change dynamically over time.

For analytical tractability, user mobility is modeled as a discrete-time process $t \in \{1, 2, 3, \dots\}$, where each user remains connected to a single access node within one time slot and transitions between nodes according to

a random-walk mobility model. This random-walk model abstracts a wide range of mobility behaviors commonly observed in MEC applications, including low-speed pedestrian movement and medium- to high-speed vehicular mobility, thereby providing a reasonable approximation of practical mobility patterns.

The traffic on a virtual link is defined as

$$\text{traffic}(e_{ji}^v) = F(G_j^V) \cdot \prod_{x=0}^i \text{ratio}(f(n_{jx}^v)) \quad (2)$$

$$\text{ratio}(f(n_{j0}^v)) = 1 \quad (3)$$

where, $\text{ratio}(\cdot)$ denotes the traffic scaling factor of a VNF.

The resource demand of a virtual node n_{ji}^v for resource type $r \in \mathcal{R}$ is:

$$\text{res}^r(n_{ji}^v) = \text{coef}^r(f(n_{ji}^v)) \cdot F(G_j^V) \cdot \prod_{x=0}^{i-1} \text{ratio}(f(n_{jx}^v)) \quad (4)$$

where, $\text{coef}^r(\cdot)$ denotes the per-unit-traffic resource coefficient.

The migration overhead of a VNF depends on its state volume:

$$V_{ji} = \sum_{r \in \mathcal{R}} \lambda^r \text{res}^r(n_{ji}^v) \quad (5)$$

where, λ^r denotes the migration rate of resource type r .

Under the pre-copy mechanism, given allocated bandwidth B_{ji} and dirty-page rate $D(f(n_{ji}^v))$, the downtime and total migration time are:

$$t_{ji}^{\text{down}} = \frac{V_{ji}}{B_{ji}} r^{N-1} \quad (6)$$

$$t_{ji}^{\text{mig}} = \frac{V_{ji}}{B_{ji}} \cdot \frac{1 - r^N}{1 - r} \quad (7)$$

where, $r = D(f(n_{ji}^v))/B_{ji}$ is the dirty-rate ratio and N denotes the number of pre-copy iterations.

For multiple VNFs, the overall downtime and migration time are obtained by the maximum over all VNFs.

The end-to-end delay of an SFC is the cumulative delay of mapped links. Its average delay over a period of T slots is:

$$d_j = \frac{1}{T} \sum_{t=1}^T d_j(t) \quad (8)$$

The service must satisfy the SLA constraint:

$$D_j \leq T_{\text{budget}}, \quad \forall j \quad (9)$$

where, T_{budget} is the maximum tolerable service delay.

Under this constraint, the problem of optimization takes into account of three objectives:

$$\begin{aligned} \min R^{\text{vnf}} &= \frac{1}{|G^V|} \sum_j \frac{N_{j,\text{mig}}^{\text{vnf}}}{N_{j,\text{total}}^{\text{vnf}}} \\ \min V^{\text{mig}} &= \frac{1}{|G^V|} \sum_j V_j^{\text{mig}} \\ \max \rho^{\text{mig}} &= \frac{N_{\text{success}}}{N_{\text{mig}}} \end{aligned} \quad (10)$$

These modeling components and optimization objectives jointly minimize delay, cost of resources, and migration overhead while ensuring SLA compliance in SFC migration under user mobility.

3 Proposed Method: Deep Reinforcement Learning-based Adaptive Migration Optimization

3.1 Graph Neural Network Module: Node Deployability Prediction

In mobility-aware scenarios, the selection of migration target nodes critically affects the delay and stability of SFCs. To enable topology-aware and intelligent migration decisions, this study formulated node deployability prediction as a binary node-classification task under fixed routing priors. Based on discrete-time network snapshots, the model predicted the probability of each physical node a feasible deployment destination, hence providing guidance for subsequent scheduling of migration.

For feature modeling, DRL-GAMO incorporates graph-structured representations derived from resource-congestion priors to jointly characterize local sufficiency of resources and neighborhood load distribution. Node features consist of normalized remaining central processing unit (CPU) and memory ratios together with the average utilization of neighboring nodes, complemented by lightweight structural indicators such as degree normalization, to enhance stability. Edge attributes preserve topological connectivity only and deliberately exclude dynamic bandwidth or path-search information to prevent high-variance features from impairing the convergence of the model. In addition, the selection of feature focuses on stable and resource-related indicators that best reflect node deployability, including remaining CPU/memory and neighbor utilization, while highly dynamic factors such as bandwidth are deferred to the Reinforcement Learning (RL) stage to ensure stable GNN convergence.

For the construction of labels, node annotations are generated from a combination of resource and congestion priors. By integrating the resource utilization of each node and its neighborhood, an overall availability score is computed and compared with an empirical threshold to distinguish deployable from non-deployable nodes. This labeling strategy omits transient factors such as path conditions and delay, to ensure temporal consistency and robustness of the supervisory signal.

The model adopted a message-passing graph neural network (GraphSAGE) to aggregate and discriminate node-level and adjacency features. Supervised training was performed with a binary cross-entropy objective over multi-slot graph snapshots, with optimization of parameters guided by validation performance. During inference, the network outputted node-level deployability probabilities, which were incorporated into the NFVI state representation to support the decision process of subsequent reinforcement-learning-based migration.

3.2 Markov Decision Process Modeling: State, Action, and Reward

In DRL-GAMO, the dynamic migration of VNFs was formulated as a MDP [21] under fixed service and routing priors. It aims to coordinate multiple objectives, including QoS assurance, downtime limitation, and migration cost control. The policy was optimized through Double Deep Q-Learning Network (DDQN), which enabled the agent to adaptively select feasible migration actions and achieve stable convergence. In each discrete time slot, the agent observed the current network and states of service, selected an action in accordance with the learned policy, triggered the pre-copy and resource reallocation processes, and updated the SFC mapping and end-to-end latency. The environment then returned an immediate reward and transitions to the next state.

State Space (S): The environmental state integrates three categories of information: (i) resource-level metrics, including link bandwidth utilization and normalized residual CPU and memory of each edge node; (ii) service-level descriptors, such as the maximum tolerable downtime defined by the SFC QoS requirements and the number of VNFs within the target SFC; (iii) structural priors learned from the GNN, represented as node-level deployability probabilities. This compact representation maintains sensitivity to availability of resources, congestion distribution, and SLA constraints, while avoiding high-variance factors introduced by online path searching or bandwidth re-optimization.

Action Space (A): To avoid combinatorial explosion and maintain the feasibility of decisions, each action is defined as a discrete combination of a continuous VNF segment and a pre-computed candidate path selected from the K -shortest path set. The agent determines migration actions according to the GNN-predicted deployability probabilities along these paths, implicitly mapping VNFs to destination nodes while satisfying CPU, memory, and bandwidth constraints.

Reward Function (R): The instantaneous reward reflects the trade-off between QoS improvement and migration overhead. If a migration is infeasible or violates hard constraints, a strong penalty is applied. Otherwise, the reward value decreases with both normalized end-to-end delay D and migration volume V , so as to balance service performance and consumption of resources:

$$R = \begin{cases} 1 - \lambda \frac{D}{T^{\text{budget}}} & N_{\text{mig}} = 0, D \leq T^{\text{budget}} \\ 1 - \left(\frac{D}{T^{\text{budget}}} \right) \left(\frac{V}{V_{\text{avg}}} \right) & N_{\text{mig}} > 0, D \leq T^{\text{budget}} \\ 0 & \text{otherwise} \end{cases} \quad (11)$$

Here $\lambda \in (0, 1)$ is a scaling coefficient that regulates the reward weight of non-migration to prevent degenerate policies [22]. This formulation unifies the QoS objectives and system constraints under a consistent optimization scale, in order to ensure interpretability and training stability.

3.3 Deep Reinforcement Learning-based Adaptive Migration Optimization Framework

As illustrated in Figure 1, the proposed DRL-GAMO framework consists of two major components: a mobility-driven network environment and a DRL agent. The environment emulates the dynamic behavior of the MEC-based NFVI, to capture user mobility, resource fluctuations, and SFC migration events, and continuously provides feedback for decision making.

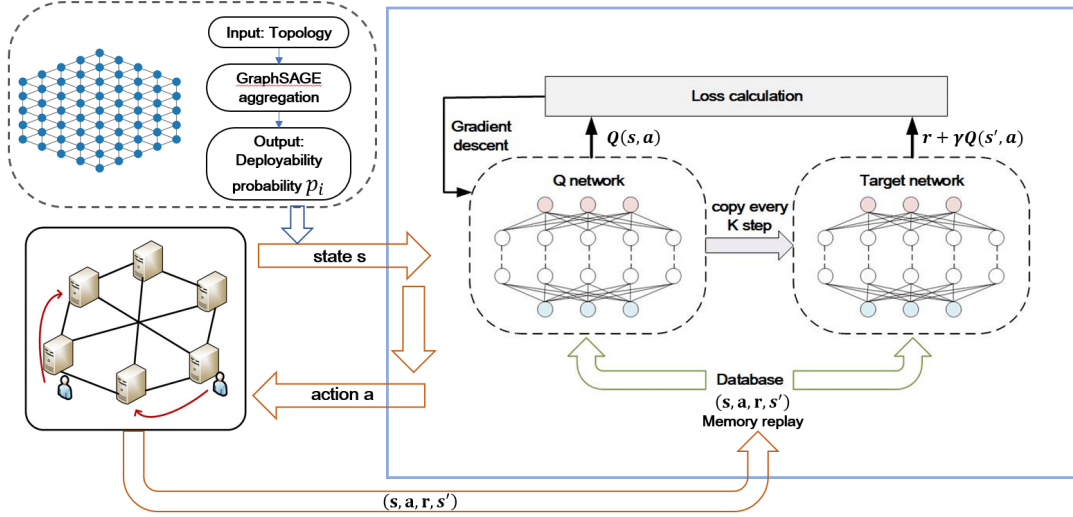


Figure 1. Framework of deep reinforcement learning-based adaptive migration optimization (DRL-GAMO)

To deliver topology-aware insights for migration decisions, the GraphSAGE module performs hierarchical message aggregation over the NFVI graph, combining node-level resource indicators (e.g., residual CPU and memory) with structural connectivity and congestion features. This process produces node-level deployability probabilities, which are integrated into the NFVI state representation as structural priors, to yield resource- and topology-sensitive states.

Building upon these enriched representations, the DDQN-based agent learns to optimize migration strategies by jointly considering predicted deployability, mobility-induced topology variations, and service-level constraints. It determines migration targets, selects feasible transmission paths, and allocates bandwidth adaptively to balance service the continuity of and migration overhead.

The interaction between the two modules follows an iterative training–execution cycle under an offline–online learning paradigm. Specifically, the GNN module is pretrained offline to extract structural priors such as node deployability; its parameters remain fixed during the subsequent DDQN training and serve only as a stable feature source, while the DDQN network is continuously updated through interaction with the environment. In the offline phase, the NFVI environment continuously generates state transitions and migration events, enabling the agent to explore and learn optimal migration policies through reinforcement learning. The learning process adopts an ϵ -greedy strategy for exploration and employs stabilized target updates to ensure convergence. Upon convergence, the trained model is deployed in the online phase, when the agent evaluates real-time NFVI states and migration demands to determine optimal migration actions. The outcomes of execution are subsequently incorporated into incremental updates, to support continual adaptation and robustness under dynamic mobility conditions.

3.4 Model Training Process

During training, the agent continuously refines its policy through interactions with the virtual network environment. At each time slot, user mobility triggers NFVI state variations and SFC migration requests, which serve as the current state. The agent selects and executes actions using the ϵ -greedy strategy, while each transition (s, a, r, s') is stored in the replay buffer. The pseudocode for the training process is presented in Algorithm 1 as follows:

Algorithm 1 DRL-GAMO Training and Execution

```
1: Input: Discount factor  $\gamma$ , learning rate  $\alpha$ , batch size  $B$ , sync period  $\tau$ , exploration rate  $\varepsilon$ , GNN module, replay buffer  $\mathcal{D}$ 
2: Output: Trained Q-network parameters  $\theta$ 
3: Initialize online Q-network  $Q(\cdot; \theta)$ , target network  $Q^-(\cdot; \theta^- \leftarrow \theta)$ , and replay buffer  $\mathcal{D}$ 
4: for each episode do
5:   Environment generates NFVI state and migration requests; construct state  $s$  (link usage, node resources, and GNN deployability)
6:   if  $\text{rand}() < \varepsilon$  then
7:     Select random action  $a$ 
8:   else
9:     Prune action space using GNN scores; select
      
$$a \leftarrow \arg \max_a Q(s, a; \theta)$$

10:   end if
11:   Execute action  $a$ , observe reward  $r$  and next state  $s'$ , store  $(s, a, r, s')$  into  $\mathcal{D}$ 
12:   Sample a mini-batch from  $\mathcal{D}$  and update  $\theta$  with the DDQN target
13:   Every  $\tau$  steps, synchronize target network:  $\theta^- \leftarrow \theta$ 
14:   Decay exploration rate  $\varepsilon$ 
15: end for
16: return  $\theta$ 
```

In our implementation, the ϵ -greedy exploration starts from $\epsilon_0 = 1.0$ and follows an exponential decay schedule $\epsilon \leftarrow \max(\epsilon \cdot 0.99995, 0.1)$ to balance exploration and exploitation during training.

For parameter updates, mini-batches are sampled to compute the target Q-value:

$$y = r + \gamma Q\left(s', \arg \max_{a'} Q(s', a'; \theta^-); \theta^-\right) \quad (12)$$

where, θ and θ^- denote the parameters of the online and target networks, respectively. The online network guides selection of action, while the target network generates target values, hence alleviating the overestimation issue of conventional DQN.

The training objective is to minimize the discrepancy between the predicted value $Q(s, a; \theta)$ and the target value y , using the Huber loss function:

$$L(\theta) = \frac{1}{B} \sum_{(s, a, r, s') \in B} l(y - Q(s, a; \theta)) \quad (13)$$

where, B denotes the sampled mini-batch of experiences and $l(\cdot)$ is the piecewise quadratic Huber loss. The explicit form of $l(\cdot)$ is given by:

$$l(x) = \begin{cases} \frac{1}{2}x^2, & |x| \leq 1, \\ |x| - \frac{1}{2}, & |x| > 1. \end{cases} \quad (14)$$

This corresponds to the standard Smooth L1 Loss used in DDQN implementation.

Given that optimization of deep reinforcement learning often involves non-stationary targets and noisy gradient updates, Adam's adaptive moment estimation provides more stable and efficient parameter updates under such conditions. Based on this theoretical suitability and preliminary verification during training, Adam shows more stable convergence behavior compared with Root Mean Square Propagation (RMSprop) and Stochastic Gradient Descent (SGD), and is therefore adopted as the optimizer for the DRL-GAMO framework.

Building on this, parameter updates in the proposed framework are carried out using the Adam optimizer:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L(\theta) \quad (15)$$

where, α is the learning rate. Meanwhile, the target network parameters are periodically synchronized from the online network to further enhance training stability.

4 Simulation Setup and Results

4.1 Experimental Environment and Parameter Settings

The experimental evaluation was conducted with the open-source simulation platform SFCSSim [23]. The test environment adopted a 61-node and 120-link hexagonal cellular topology that included access, aggregation, and core

layers, as illustrated in Figure 2. This topology effectively emulated SFC deployment and VNF migration processes in MEC scenarios. SFCSim provides built-in support for VNF deployment, resource accounting, and pre-copy-based migration modeling, which enables seamless integration with our DRL training pipeline and ensures consistent and reproducible experimental settings.

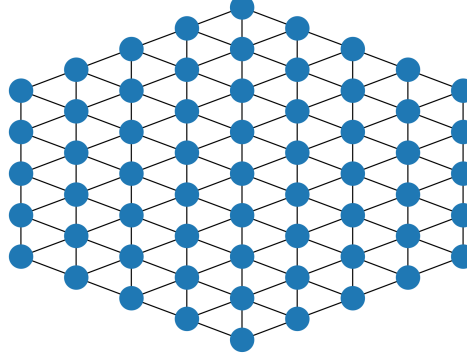


Figure 2. Cellular network topology

The detailed simulation parameters, including NFVI resources, VNF configurations, and SFC characteristics, are listed in Table 1. User mobility follows a random-walk model with a period of 10–20 minutes. At each discrete time slot, predefined handovers between adjacent access nodes update the SFC source node, thereby triggering migration events whenever necessary. The DRL agent adopts a five-layer fully connected Q-network with 256 neurons per layer and rectified linear unit (ReLU) activation.

Table 1. Simulation parameters

Category	Parameter	Value
Network function virtualization infrastructure (NFVI)	Link bandwidth	8 Gbps
	Propagation delay	0.5 ms
	Node resources	20 cores/64 GB
	Type	8
Virtual network function (VNF)	Traffic scaling factor	$U(0.8, 1.2)$
	CPU resource coefficient	0.02–0.03 cores/Mbps
	Memory resource coefficient	80–100 MB/Mbps
	Dirty page rate	50 MBps
	Traffic demand	$U(10, 20)$ Mbps
	Latency requirement	5–7 ms
Service function chain (SFC)	Required NFs	4–6
	Lifetime	1–3 time slots
	Mobility model	Random walk
	Max downtime	1 s
	Migration period	10–20 min

Note: 1. $U(a, b)$ denotes a uniform distribution between a and b . 2. Node resources indicate the maximum CPU cores and memory per edge node.

Before determining the final training configuration, a sensitivity analysis of the learning rate was conducted to assess its impact on convergence stability and training efficiency. The corresponding results are shown in Figure 3, where a learning rate of 3×10^{-5} exhibits the most stable and effective convergence among all tested candidates, and is therefore selected as the final learning rate. The discount factor is set to $\gamma = 0.95$.

For benchmarking, four baseline algorithms were considered:

- **PLARBA**: Estimates migration time under the pre-copy mechanism and selects the shortest feasible migration path.
- **PPLARBA**: Extends PLARBA by enabling parallel migration of multiple VNFs, hence reducing overall migration time.
- **DRL-ADMO**: Identifies VNFs which require migration by comparing consecutive deployments and uses DRL to learn migration paths and target nodes.

- **FMC**: Selects low-cost paths by combining K -shortest paths with a similarity-based metric to improve the continuity of service.

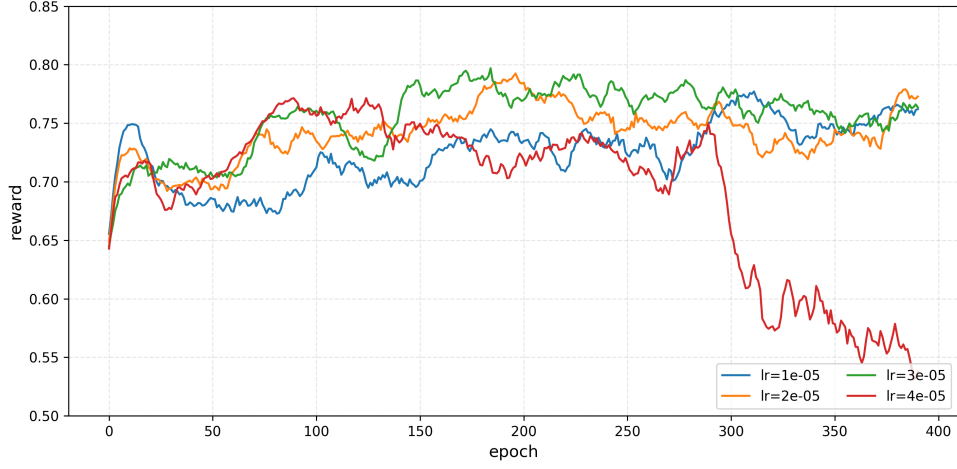


Figure 3. Comparison of different learning rates

4.2 Experimental results

To provide robust prior information on node deployability for the selection of migration actions, this study first conducted independent training on a Graph Neural Network (GNN) module. Taking key features such as node resource margin, neighborhood load, and local topological structure as inputs, the module predicted whether a node was suitable as a migration destination. Its output serves as an important structured prior in the reinforcement learning phase, thereby improving the efficiency of action screening and the accuracy of feasibility evaluation.

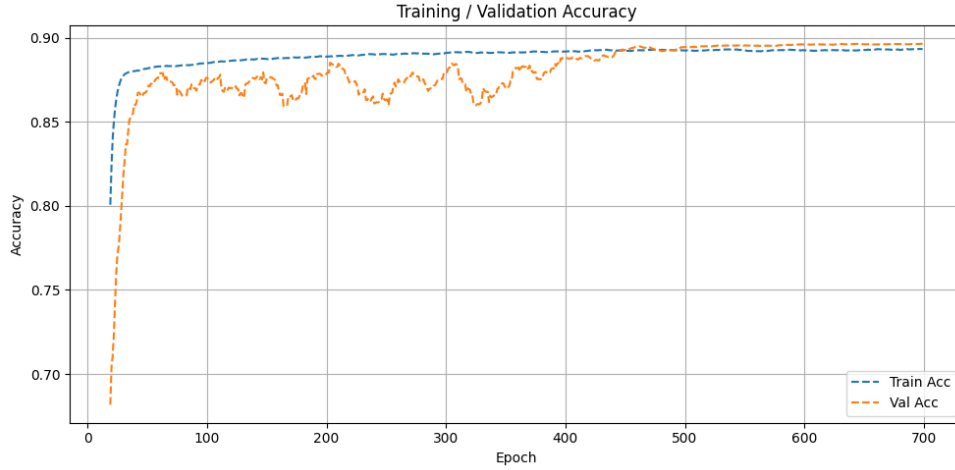


Figure 4. Training and validation accuracy curves

As shown in Figure 4, the GraphSAGE model exhibits clear accuracy improvement on both the training and validation sets. The accuracy rises rapidly from below 0.7 to approximately 0.95 within the first 50 epochs and becomes stable after around 100 epochs. Ultimately, the training accuracy stabilizes around 0.985, while the validation accuracy remains slightly higher at approximately 0.99. These results indicate that GraphSAGE effectively captures structural correlations between nodes and their neighborhoods and demonstrates strong ability of generalization without significant overfitting.

Figure 5 presents the loss curves during model training. Both the training and validation losses decrease rapidly at the early stage and eventually stabilize within the range of 0.25–0.30. The curves follow a consistent trend with no noticeable fluctuations, indicating stable optimization and effective parameter updates. Overall, GraphSAGE successfully captures joint resource–topology features, thus providing reliable prior knowledge for subsequent evaluation of migration feasibility.

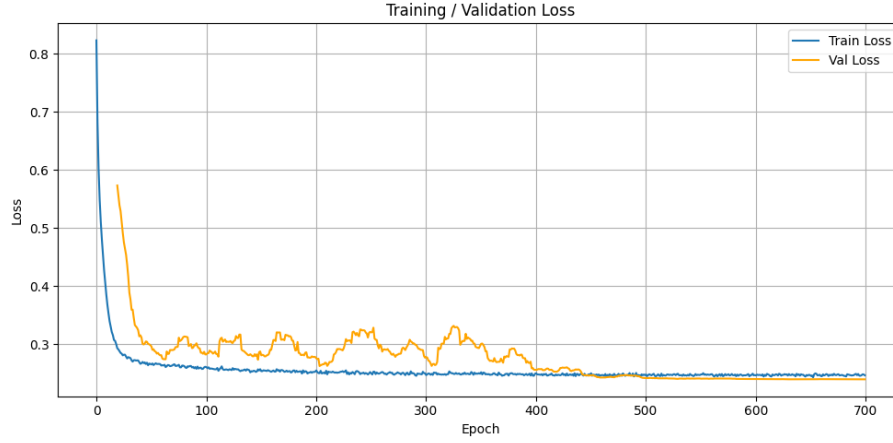


Figure 5. Training and validation loss curves

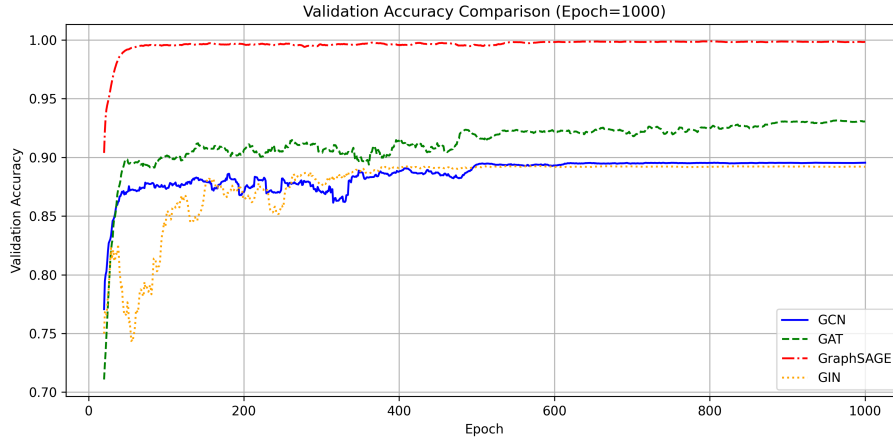


Figure 6. Validation accuracy comparison of different graph neural network (GNN) models

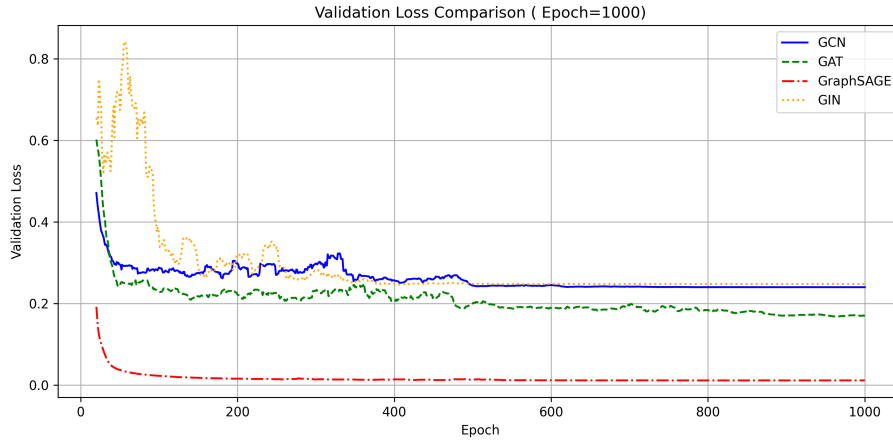


Figure 7. Validation loss comparison of different graph neural network (GNN) models

In addition, this study compared GraphSAGE with three representative GNN models: Graph Convolutional Networks (GCN), Graph Attention Networks (GAT), and Graph Isomorphism Network (GIN). As shown in Figure 6, GraphSAGE achieved the highest and most stable validation accuracy, approaching 1.0 after convergence, while GAT, GCN, and GIN converged to approximately 0.90, 0.88, and 0.87, respectively. Figure 7 further shows that GraphSAGE attained the lowest validation loss (around 0.05) and the fastest convergence rate. These results confirm its superior feature extraction capability and generalization performance in complex topologies. Therefore,

GraphSAGE is adopted as the final node evaluation model to provide reliable priors for subsequent reinforcement learning optimization.

After completing GNN pre-training, the DDQN training process was further evaluated under different values of K , the number of candidate migration paths, to analyze the relationship between action-space size and learning stability.

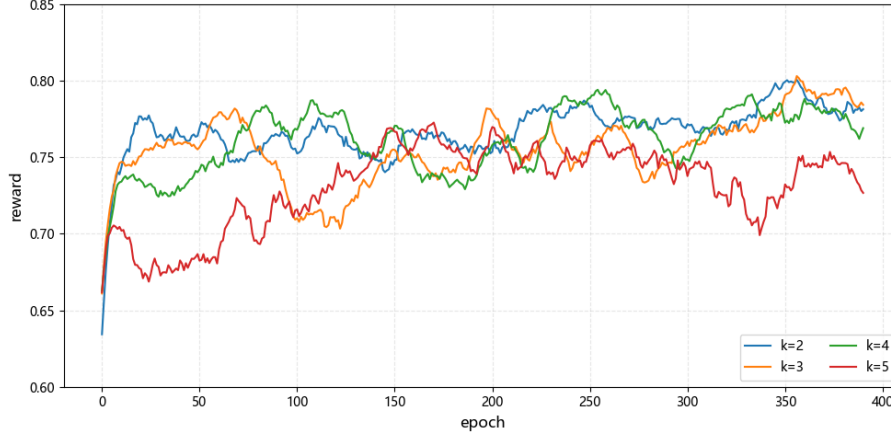


Figure 8. Reward convergence curves under different K values

Figure 8 presents the reward curves corresponding to different values of K . All curves exhibit the typically “rapid rise–oscillation–stabilization” convergence pattern, indicating that the agent can quickly acquire basic migration strategies during the exploration phase and gradually converge thereafter. When $K = 2$ and $K = 3$, the reward curves converge the fastest, with final average reward values reaching approximately 0.78–0.80, thus demonstrating that a moderate number of candidate paths achieves a good balance between action flexibility and training stability. In contrast, when $K = 4$, the oscillation amplitude becomes significantly larger; and when $K = 5$, the fluctuations are even more pronounced with a lower final reward. This suggests that an excessively large action space substantially increases the difficulty of policy exploration and degrades the convergence quality.

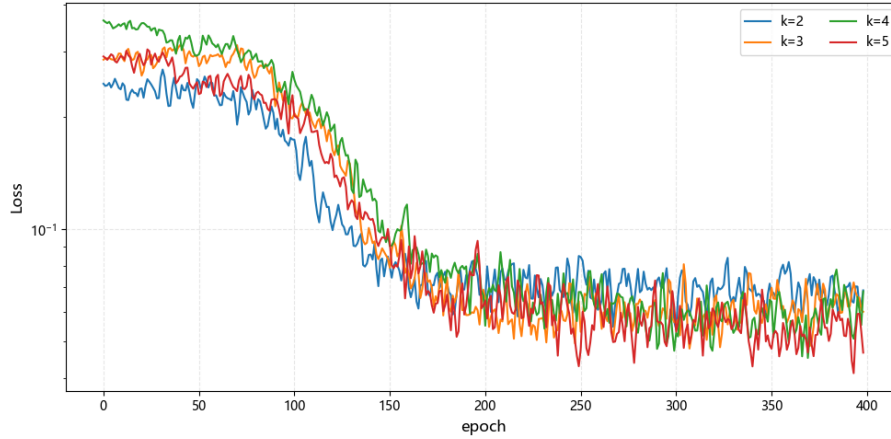


Figure 9. Loss curves under different K values

Figure 9 shows the loss curves for different values of K . Overall, the losses under all settings stabilized after approximately 150–200 steps and eventually converged to the order of 10^{-2} , indicating good overall training stability. In terms of convergence speed and smoothness, the cases $K = 2$ and $K = 3$ exhibit the most stable behavior with minimal oscillation. In contrast, the curves for $K = 4$ and $K = 5$ display significantly higher volatility, reflecting increased instability during optimization. These results further confirm that adopting a moderate candidate path size (e.g., $K = 3$) effectively accelerates policy learning while enhancing overall stability.

The performance of different algorithms was first compared under varying traffic loads, as shown in Figure 10a–d. Overall, the proposed DRL-GAMO consistently outperforms baseline approaches by achieving higher migration success rates and lower migration overhead, while keeping service delay within SLA requirements.

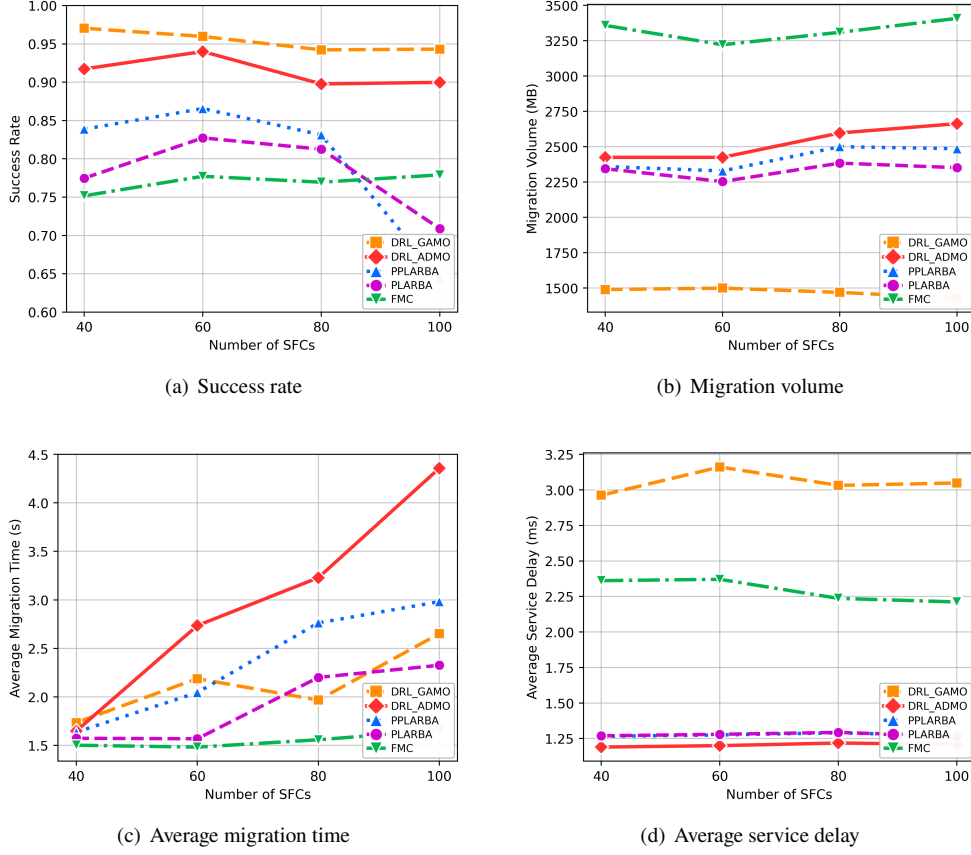


Figure 10. Algorithm performance under different service function chain (SFC) scales

Figure 10a shows that as the number of SFCs increases, the success rates of PLARBA and PPLARBA drop sharply (below 70% under heavy load), reflecting limited adaptability in dynamic environments. DRL-ADMO remained stable at around 90%, while DRL-GAMO consistently exceeded 95%, reaching nearly 96% when the number of SFCs equaled 100, thus demonstrating stronger robustness. Figure 10b shows the migration volume: FMC incurred the highest overhead (above 3000 MB), PLARBA and PPLARBA remained around 2400 MB, and DRL-ADMO slightly exceeded 2500 MB. By contrast, DRL-GAMO achieved the lowest migration volume (about 1500 MB under heavy load), 40% lower than DRL-ADMO, significantly reducing the consumption of resources while ensuring service continuity. Figure 10c illustrates the migration time: PLARBA and PPLARBA gradually increased to approximately 2.5–3 s, while DRL-ADMO surged above 4 s under heavy load. DRL-GAMO grew the slowest, remaining at about 2.7 s for 100 SFCs, roughly 1.8 s shorter than DRL-ADMO, thus highlighting its advantage in reducing migration delay and mitigating service-interruption risks. Figure 10d presents the average service delay: PLARBA and PPLARBA stayed at around 1.2 ms, DRL-ADMO achieved about 1.3 ms, and DRL-GAMO yielded slightly higher values but always below 3.2 ms, remaining well within the 5–7 ms SLA constraint. The slightly higher latency of DRL-GAMO is mainly due to its optimization preference for reducing migration overhead. The agent tends to select strategies involving fewer VNF relocations and smaller migration volume, even if the corresponding paths are marginally longer. This leads to a small latency increase but significantly lowers migration cost and enhances overall robustness. Despite this increase, the resulting latency always remains well below the SLA requirement across all SFC scales, indicating that the slight growth in delay is acceptable and does not affect the quality of service.

These results confirm that DRL-GAMO achieves the best balance between reliability and efficiency: it improves success rate, reduces migration resource consumption and migration time, and keeps service delay within acceptable limits. Having compared different algorithms, the impact of candidate path numbers were further analyzed.

Figure 11a–d illustrates the performance trends of DRL-GAMO under different candidate path counts (K) as the number of SFCs increases. Overall, as the load grows from 40 to 100 SFCs, intensified competition for resources reduces migration success rate, lengthens migration time and service latency, and lowers consumption of migration resources, as the system tends to avoid unnecessary migrations under heavy load. Clear trade-offs emerge with K : larger K improves the success rate but increases migration time and latency fluctuations.

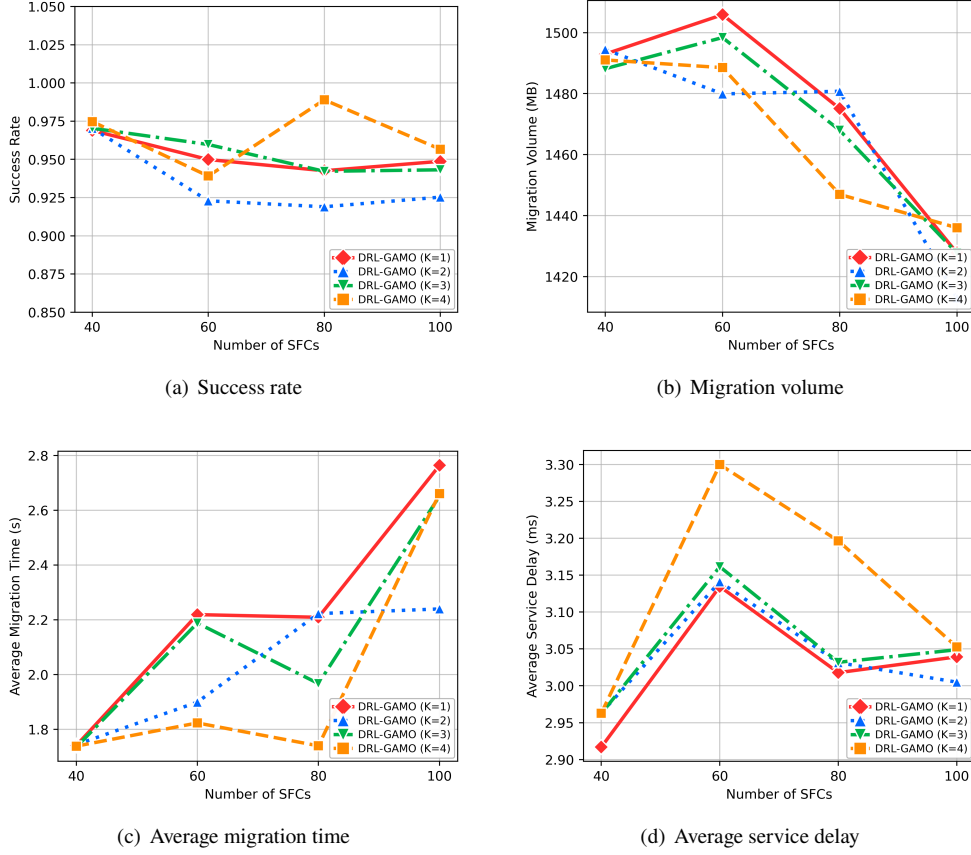


Figure 11. Deep reinforcement learning–based adaptive migration optimization (DRL-GAMO) performance under different candidate path numbers K

Figure 11a shows that migration success rate decreases slightly with larger SFC scales, but $K = 4$ consistently achieves the highest level, exceeding 95% at 100 SFCs and outperforming $K = 2$ by about 3–4% to demonstrate stronger robustness. Figure 11b shows that migration resource consumption decreases as the number of SFCs increases; under heavy load, $K = 2$ and $K = 3$ are noticeably more efficient than $K = 1$, with $K = 2$ performing best at 100 SFCs. Figure 11c shows that migration time continuously increases: $K = 1$ is always the highest, $K = 2$ and $K = 3$ grow more smoothly, while $K = 4$ remains the shortest under light load but increases rapidly under heavy load (e.g., when the number of SFCs ≥ 80), reaching values of 20% higher than $K = 2$. Figure 11d presents the average service delay: latency gradually increases but remains below 3.3 ms in all cases, meeting SLA requirements. Among all settings, $K = 2$ consistently achieved the lowest and most stable latency, whereas $K = 4$ exhibited noticeable fluctuations under medium load.

In summary, with increasing SFC numbers, system performance follows clear patterns: success rate and migration resource consumption decrease, while migration time and latency increase. A moderate number of candidate paths ($K = 2$ –3) provides the best trade-off between migration overhead and overall performance, whereas $K = 4$ further improves the success rate at the cost of higher migration time and latency, hence demonstrating stronger robustness.

5 Conclusions

This paper proposed DRL-GAMO, a graph-enhanced deep reinforcement learning framework for dynamic SFC migration in MEC environments. By integrating a GNN for topology-aware node deployability prediction with a DDQN for adaptive migration decision-making, DRL-GAMO effectively balanced service latency and resource overhead during migration. The proposed approach enabled intelligent VNF selection, resource-efficient path planning, and adaptive scheduling under dynamic network conditions.

Extensive simulations on the SFCSim platform demonstrate that DRL-GAMO consistently achieved higher migration success rates and lower migration time and volume compared with existing algorithms such as PLARBA, PPLARBA, and DRL-ADMO, while maintaining service latency within strict QoS constraints. Under heavy traffic and mobility, DRL-GAMO exhibited strong robustness and adaptability, sustaining stable service performance even in resource-constrained scenarios. These results confirm its potential as a practical and scalable solution for efficient SFC migration and orchestration in future MEC networks.

Author Contributions

Conceptualization, M.L. and H.H.; methodology, M.L.; investigation, M.L.; validation, M.L.; data curation, M.L.; writing—original draft preparation, M.L.; writing—review and editing, H.H.; visualization, M.L.; supervision, H.H.; resources, H.H.; funding acquisition, H.H.; formatting check, J.R. All authors have read and agreed to the published version of the manuscript.

Data Availability

The data used to support the research findings are available from the corresponding author upon request.

Conflicts of Interest

The authors declare no conflict of interest.

References

- [1] D. Bhamare, R. Jain, M. Samaka, and A. Erbad, “A survey on service function chaining,” *J. Netw. Comput. Appl.*, vol. 75, pp. 138–155, 2016. <https://doi.org/10.1016/j.jnca.2016.09.001>
- [2] J. Gil Herrera and J. F. Botero, “Resource allocation in NFV: A comprehensive survey,” *IEEE Trans. Netw. Serv. Manag.*, vol. 13, no. 3, pp. 518–532, 2016. <https://doi.org/10.1109/TNSM.2016.2598420>
- [3] K. Kaur, V. Mangat, and K. Kumar, “A review on virtualized infrastructure managers with management and orchestration features in NFV architecture,” *Comput. Netw.*, vol. 217, p. 109281, 2022. <https://doi.org/10.1016/j.comnet.2022.109281>
- [4] Y. Zhang, R. Wang, J. Hao, Q. Wu, Z. Xiong, and D. Niyato, “Joint deployment and migration of service function chains for mobility-aware services in an edge-cloud environment,” *IEEE Trans. Cogn. Commun. Netw.*, pp. 237–250, 2025. <https://doi.org/10.1109/TCCN.2025.3557966>
- [5] G. Mirjalily and Z. Luo, “Optimal network function virtualization and service function chaining: A survey,” *Chin. J. Electron.*, vol. 27, no. 4, pp. 704–717, 2018. <https://doi.org/10.1049/cje.2018.05.008>
- [6] Y. T. Chen and W. J. Liao, “Mobility-aware service function chaining in 5G wireless networks with mobile edge computing,” in *ICC 2019-2019 IEEE International Conference on Communications (ICC)*, 2019, pp. 1–6. <https://doi.org/10.1109/ICC.2019.8761306>
- [7] B. Yi, X. Wang, M. Huang, and K. Li, “Design and implementation of network-aware VNF migration mechanism,” *IEEE Access*, vol. 8, pp. 44 346–44 358, 2020. <https://doi.org/10.1109/ACCESS.2020.2978002>
- [8] H. Hu, C. Yang, L. Xu, T. Song, and B. B. Dalia, “Path load adaptive migration for routing and bandwidth allocation in mobile-aware service function chains,” *Electronics*, vol. 11, p. 57, 2022. <https://doi.org/10.3390/electronics11010057>
- [9] L. Tang, T. Wu, X. Zhou, and Q. Chen, “A virtual network function migration algorithm based on federated learning prediction of resource requirements,” *J. Electron. Inf. Technol.*, vol. 44, no. 10, pp. 3532–3540, 2022. <https://doi.org/10.11999/JEIT210743>
- [10] S. Guo, L. Liu, T. Jing, and H. Liu, “SFC active reconfiguration based on user mobility and resource demand prediction in dynamic IoT-MEC networks,” *PLoS ONE*, vol. 19, no. 8, p. e0306777, 2024. <https://doi.org/10.1371/journal.pone.0306777>
- [11] Z. Liao, W. Deng, S. He, and Q. Tang, “Collaborative filtering-based fast delay-aware algorithm for joint VNF deployment and migration in edge networks,” *Comput. Netw.*, vol. 243, p. 110300, 2024. <https://doi.org/10.1016/j.comnet.2024.110300>
- [12] H. Qu, K. Wang, and J. Zhao, “Priority-aware VNF migration method based on deep reinforcement learning,” *Comput. Netw.*, vol. 208, p. 108866, 2022. <https://doi.org/10.1016/j.comnet.2022.108866>
- [13] H. Liu, J. Chen, J. Chen, X. Cheng, K. Guo, and Y. Qin, “A deep Q-learning based VNF migration strategy for elastic control in SDN/NFV networks,” in *2021 International Conference on Wireless Communications and Smart Grid (ICWCSG)*, 2021, pp. 217–223. <https://doi.org/10.1109/ICWCSG53609.2021.00049>
- [14] L. Xu, W. Liu, Z. Wang, H. Zhang, Z. Wang, Y. Guo, and T. Song, “Mobile-aware SFC intelligent seamless migration in multi-access edge computing,” *J. Netw. Syst. Manag.*, vol. 32, p. 49, 2024. <https://doi.org/10.1007/s10922-024-09820-0>
- [15] J. L. Vieira, E. L. C. Macedo, A. L. E. Battisti, J. Noce, P. F. Pires, D. C. Muchaluat-Saade, A. C. B. Oliveira, and F. C. Delicato, “Mobility-aware SFC migration in dynamic 5G edge networks,” *Comput. Netw.*, vol. 250, p. 110571, 2024. <https://doi.org/10.1016/j.comnet.2024.110571>
- [16] D. Liu, Z. Zhou, D. Zhang, K. Guo, Y. Wu, and C. Wu, “Efficient service reconfiguration with partial virtual network function migration,” *Comput. Netw.*, vol. 241, p. 110205, 2024. <https://doi.org/10.1016/j.comnet.2024.110205>

- [17] Y. Zhang and J. Zhao, “Adaptive service function chain migration in satellite–terrestrial integrated networks,” in *2025 IEEE/CIC International Conference on Communications in China (ICCC)*, 2025, pp. 1–6. <https://doi.org/10.1109/ICCC65529.2025.11148775>
- [18] K. Qu, W. Zhuang, X. Shen, X. Li, and J. Rao, “Dynamic resource scaling for VNF over nonstationary traffic: A learning approach,” *IEEE Trans. Cogn. Commun. Netw.*, vol. 7, no. 2, pp. 648–662, 2021. <https://doi.org/10.1109/TCCN.2020.3018157>
- [19] T. W. Kuo, B. H. Liou, K. C. Lin, and M. J. Tsai, “Deploying chains of VNFs: On the relation between link and server usage,” *IEEE/ACM Trans. Netw.*, vol. 26, no. 4, pp. 1562–1576, 2018. <https://doi.org/10.1109/TNET.2018.2842798>
- [20] P. Sun, J. Lan, J. Li, Z. Guo, and Y. Hu, “Combining deep reinforcement learning with graph neural networks for optimal VNF placement,” *IEEE Commun. Lett.*, vol. 25, no. 1, pp. 176–180, 2020. <https://doi.org/10.1109/LCOMM.2020.3025298>
- [21] Y. Li, P. Zhang, N. Kumar, M. Guizani, J. Wang, K. I. Kostromitin, Y. Wang, and L. Tan, “Reliability-assured service function chain migration strategy in edge networks using deep reinforcement learning,” *J. Netw. Comput. Appl.*, vol. 231, p. 103999, 2024. <https://doi.org/10.1016/j.jnca.2024.103999>
- [22] L. Xu, H. Hu, and Y. Liu, “SFCSim: A network function virtualization resource allocation simulation platform,” *Cluster Comput.*, vol. 26, no. 1, pp. 423–436, 2023. <https://doi.org/10.1007/s10586-022-03670-8>
- [23] K. Huang and C. Chen, “Subgraph generation applied in GraphSAGE to deal with imbalanced node classification,” *Soft Comput.*, vol. 28, no. 17–18, pp. 10 727–10 740, 2024. <https://doi.org/10.1007/s00500-024-09797-7>

Nomenclature

$G^S = (N^S, E^S)$	NFVI graph consisting of physical nodes and links
N^S	Set of physical nodes
E^S	Set of physical links
b_{ij}	Bandwidth of physical link (i, j) , $\text{MB}\cdot\text{s}^{-1}$
d_{ij}	Link delay of physical link (i, j) , ms
P_k	Candidate migration path k between source and destination
$D(P_k)$	End-to-end delay on path P_k , ms
λ	SFC traffic demand, Mbps
α_i	Traffic scaling factor of VNF i (in/out rate ratio)
$\rho(i, r)$	Resource coefficient of VNF i for resource type r
M_i	Migration data volume of VNF i , MB
B_m	Allocated migration bandwidth, $\text{MB}\cdot\text{s}^{-1}$
T_{mig}	Total migration time, s
T_{down}	Service downtime during migration, s
τ_{SFC}	Average end-to-end service delay, ms
τ_{max}	SLA delay threshold, ms

Greek symbols

γ	Discount factor in the reinforcement learning algorithm
θ	Trainable parameters of the Q-network

Subscripts

i	Index of virtual network function (VNF)
j	Index of destination node or linked node
s, d	Source and destination nodes in an SFC path
n	Index of physical node
r	Type of resource (CPU, memory, bandwidth)
t	Time slot or decision step
k	Index of candidate migration path